PULMO · AI

Détection du cancer pulmonaire par fusion multimodale

Machine Learning + Deep Learning + MLOps

TP Noté · Module : Intelligence Artificielle & Machine Learning

Formation : M2 ESIC — Année 2025-2026

Professeur : Redouane FENZI

Étudiant : Abderrahim LOUDIYI

Date : Avril 2026

Pipeline complet : EDA · ML tabulaire · CNN fine-tuné · Fusion multimodale · FastAPI · Next.js · Docker · Render · Vercel

Sommaire

1. Introduction et objectifs
2. Description des données
3. Partie 0 — Analyse exploratoire (EDA)
4. Partie 1 — Modèle tabulaire (3 classes de risque)
5. Partie 2 — CNN image + Fusion multimodale
6. Partie 3 — Analyse et interprétation
7. Partie 4 — MLOps : Architecture et déploiement
8. Technologies et liens utiles
9. Instructions de lancement et de déploiement
10. Conclusion

1. Introduction et objectifs

Ce rapport présente la conception, l'entraînement et le déploiement d'un système d'aide à la décision pour la détection du cancer pulmonaire à partir de deux modalités complémentaires : des **données cliniques tabulaires** (14 variables patient) et des **radiographies thoraciques** (184 images de la base JSRT).

L'objectif est de construire un pipeline **ML + DL + MLOps** complet, composé de deux modèles interconnectés :

- **Modèle 1** : classification tabulaire du risque de malignité en 3 classes (faible / intermédiaire / élevé).
- **Modèle 2** : classification binaire de la présence probable d'un cancer (image seule vs fusion multimodale image + probabilités du Modèle 1).
- **MLOps** : exposition des modèles via une API REST, intégration dans une interface utilisateur web, conteneurisation Docker et déploiement sur le cloud.

PATIENTS	VARIABLES	RADIOS	CLASSES IMAGE
184	**14**	**184**	**3**

2. Description des données

2.1 Jeu de données tabulaire

Le fichier `patients_cancer_poumon.csv` contient 184 patients et 20 colonnes. Toutes les données sont complètes (aucune valeur manquante). Les 14 variables utilisées comme features sont :

Variable	Type	Description
age	int	Âge du patient (21 à 80 ans)
sexe_masculin	bool	1 = homme, 0 = femme
presence_nodule	bool	Nodule détecté à la radiographie
subtilite_nodule	int (1-5)	Difficulté de détection du nodule
taille_nodule_px	int	Taille approximative en pixels
x_nodule_norm, y_nodule_norm	float (0-1)	Position normalisée du nodule
tabagisme_paquets_annee	float	Consommation cumulée (paquets-années)
toux_chronique, dyspnee, douleur_thoracique, perte_poids	bool	Symptômes cliniques
spo2	int	Saturation en oxygène (%)
antecedent_familial	bool	Antécédent familial de cancer

Les deux cibles à prédire sont :

- `risque_malignite` (3 classes : 0 = faible, 1 = intermédiaire, 2 = élevé) → **Modèle 1**
- `cancer_image` (binaire : 0 = non probable, 1 = probable) → **Modèle 2**

2.2 Radiographies JSRT

Les 184 radiographies thoraciques proviennent d'un sous-ensemble de la base **JSRT** (Japanese Society of Radiological Technology), réparties dans 3 dossiers selon leur classe de diagnostic :

Classe	Nombre	Proportion
Sain	30	16.3%
Bénin	54	29.3%
Malin	100	54.4%

3. Partie 0 — Analyse exploratoire (EDA)

L'analyse exploratoire a été réalisée avec `pandas`, `matplotlib` et `seaborn` dans le script `src/eda.py`. Aucune valeur manquante, toutes les variables sont complètes sur les 184 patients.

3.1 Distribution des classes cibles

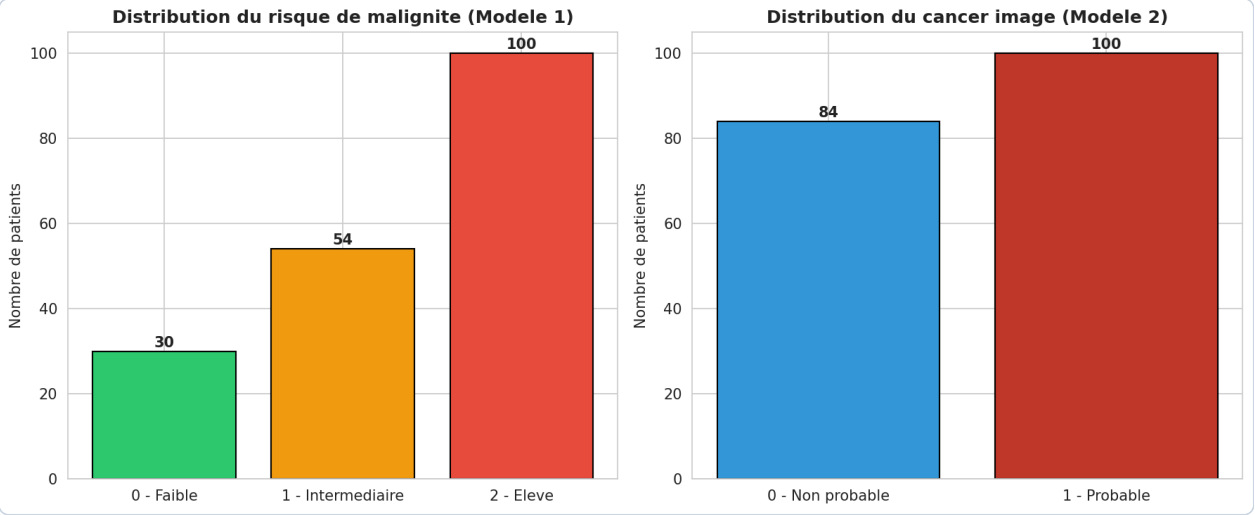


Figure 1 — Distribution des deux cibles : risque de malignité (3 classes) et cancer image (binaire)

Les classes sont déséquilibrées : le risque élevé domine (100 patients), suivi de l'intermédiaire (54) et du faible (30). Pour la cible binaire, 100 patients ont un cancer probable contre 84 non probable.

3.2 Variables cliniques par niveau de risque

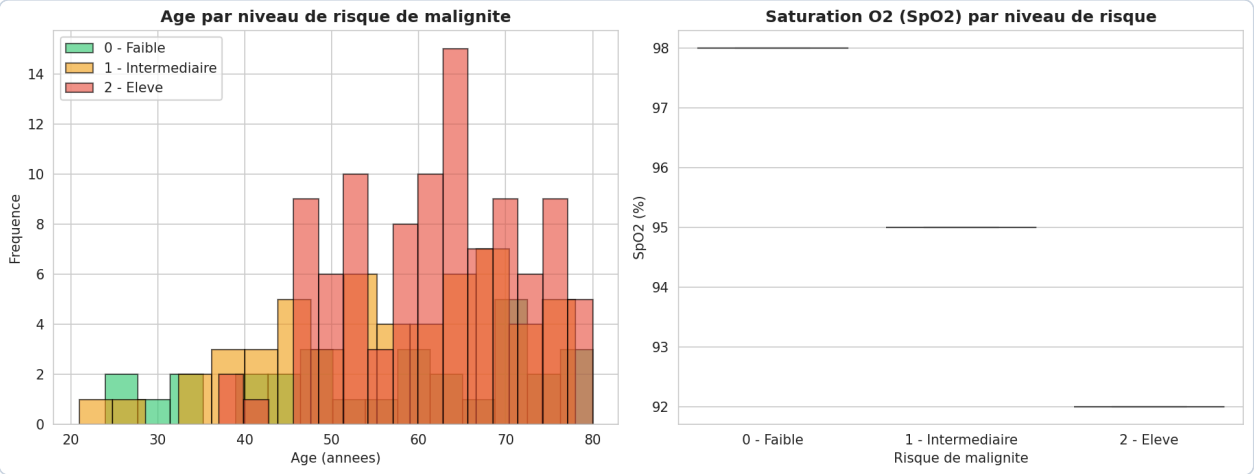


Figure 2 — Distribution de l'âge et de la SpO_2 par niveau de risque

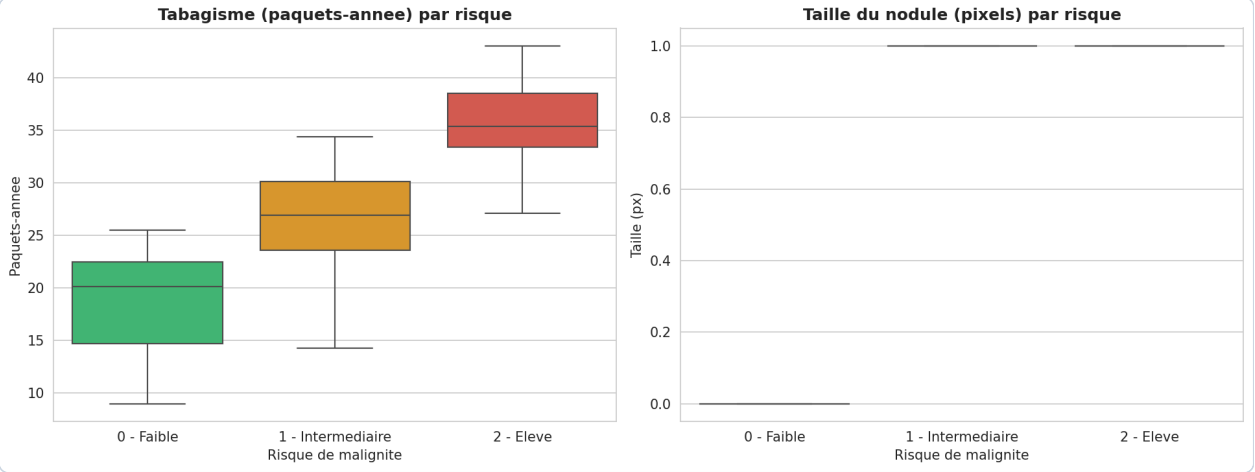


Figure 3 — Tabagisme (paquets-années) et taille du nodule par niveau de risque

On observe des séparations parfaites entre les classes pour plusieurs variables, en particulier **SpO_2** et **tabagisme**. Ceci constitue un premier indice fort que les données sont **synthétiquement générées avec des corrélations artificielles** — point discuté dans la Partie 3.

3.3 Matrice de corrélation

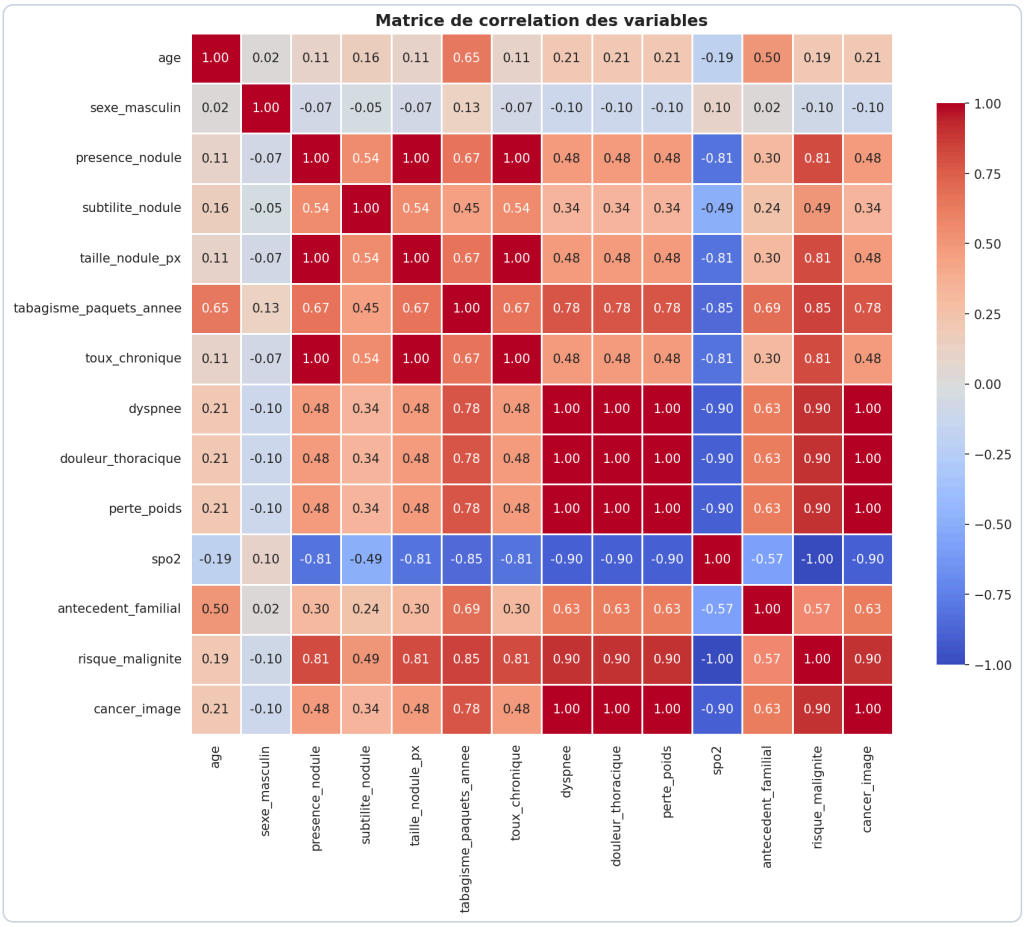


Figure 4 — Matrice de corrélation entre les variables tabulaires

Les symptômes cliniques (toux, dyspnée, douleur, perte de poids) sont fortement corrélés entre eux et avec la cible `risque_malignite` (coefficients > 0.8). Le tabagisme et la SpO_2 ont également une corrélation > 0.9 avec la classe cible.

3.4 Radios représentatives

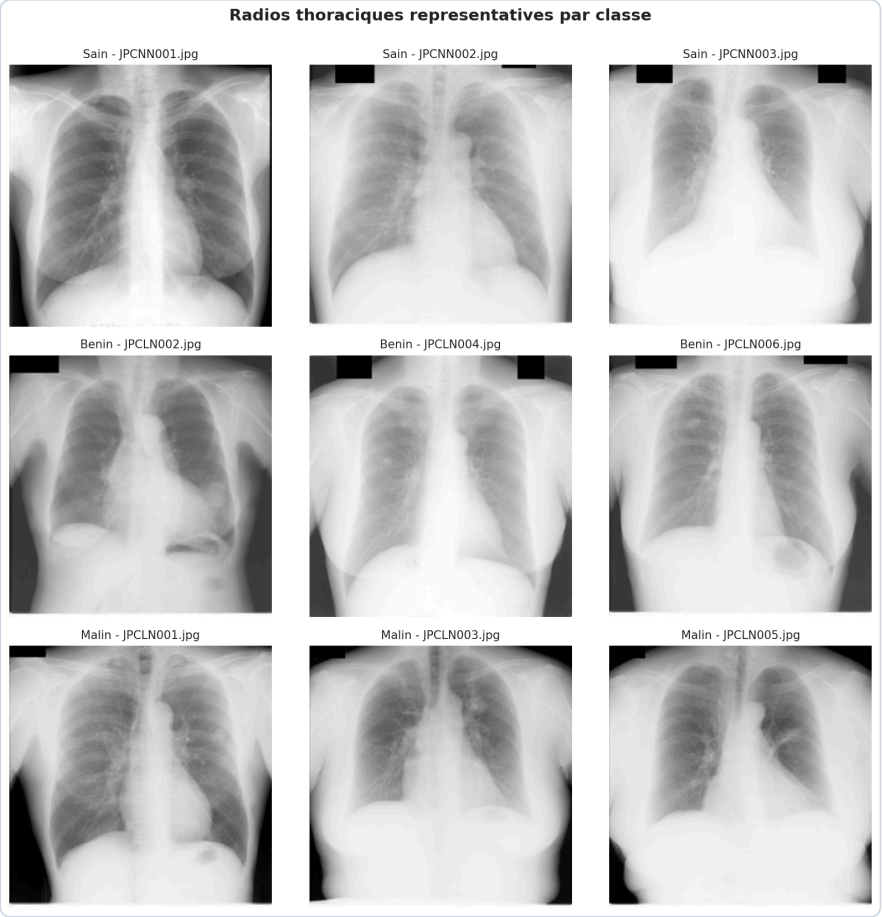


Figure 5 — Trois exemples de radiographies thoraciques par classe

3.5 Prétraitement des données

Le prétraitement a été réalisé en deux pipelines distincts, un pour les données tabulaires, un pour les images.

Données tabulaires

- **Pas de nettoyage nécessaire** : aucune valeur manquante, aucun doublon, aucun outlier aberrant détecté.
- **Normalisation** via `StandardScaler` de scikit-learn, intégré dans un `Pipeline` pour éviter toute fuite de données (scaler fitté uniquement sur le train).
- **Split stratifié 80/20** sur la cible `risque_malignite` afin de préserver la distribution des 3 classes dans le train et le test.
- **Validation croisée stratifiée 5-fold** pour évaluer la stabilité des modèles.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, StratifiedKFold

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000, multi_class="multinomial")),
])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Images radiographiques

- **Redimensionnement** : toutes les radios sont resamplées à **160 × 160 pixels** (compromis entre résolution et mémoire du batch).
- **Conversion RGB** : les radios originales sont en niveaux de gris, dupliquées sur 3 canaux pour être compatibles avec MobileNetV2 pré-entraîné sur ImageNet.
- **Normalisation ImageNet** : application de `preprocess_input` de MobileNetV2 qui centre les pixels dans l'intervalle `[-1, 1]`.
- **Data augmentation** (uniquement en train) : flip horizontal, rotation ±15%, zoom ±20%, variation de contraste ±20%, translation ±10% — indispensable étant donné le très faible volume de données (147 images d'entraînement).
- **Class weights équilibrés** calculés via `sklearn.utils.class_weight.compute_class_weight("balanced")` pour compenser le léger déséquilibre entre les deux classes de la cible binaire.

```
from PIL import Image
import numpy as np
from tensorflow.keras import layers
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

def load_image(path, size=160):
    img = Image.open(path).convert("RGB").resize((size, size))
    return np.array(img, dtype=np.float32)

X_img = np.array([load_image(p) for p in image_paths])
X_img = preprocess_input(X_img)   # normalisation ImageNet → [-1, 1]

data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.2),
    layers.RandomContrast(0.2),
    layers.RandomTranslation(0.1, 0.1),
])

class_weights = compute_class_weight("balanced",
                                     classes=np.array([0, 1]),
                                     y=y_train)
```

4. Partie 1 — Modèle tabulaire (3 classes)

Trois algorithmes de classification ont été comparés sur la tâche de prédiction du **risque de malignité en 3 classes** :

- **Logistic Regression** (multinomiale, avec normalisation StandardScaler)
- **Random Forest** (200 arbres, class_weight="balanced")
- **Gradient Boosting** (200 arbres)

4.1 Protocole expérimental

- Split stratifié 80 / 20 (147 train / 37 test)
- Normalisation via `StandardScaler` dans un `Pipeline` scikit-learn
- Validation croisée stratifiée 5-fold sur le train
- Métriques : accuracy, F1 macro, F1 weighted, matrice de confusion

4.2 Résultats

Modèle	CV Accuracy	Test Accuracy	Test F1 macro	Test F1 weighted
Logistic Regression	1.0000 (±0.0000)	**1.0000**	**1.0000**	**1.0000**
Random Forest	1.0000 (±0.0000)	1.0000	1.0000	1.0000
Gradient Boosting	1.0000 (±0.0000)	1.0000	1.0000	1.0000

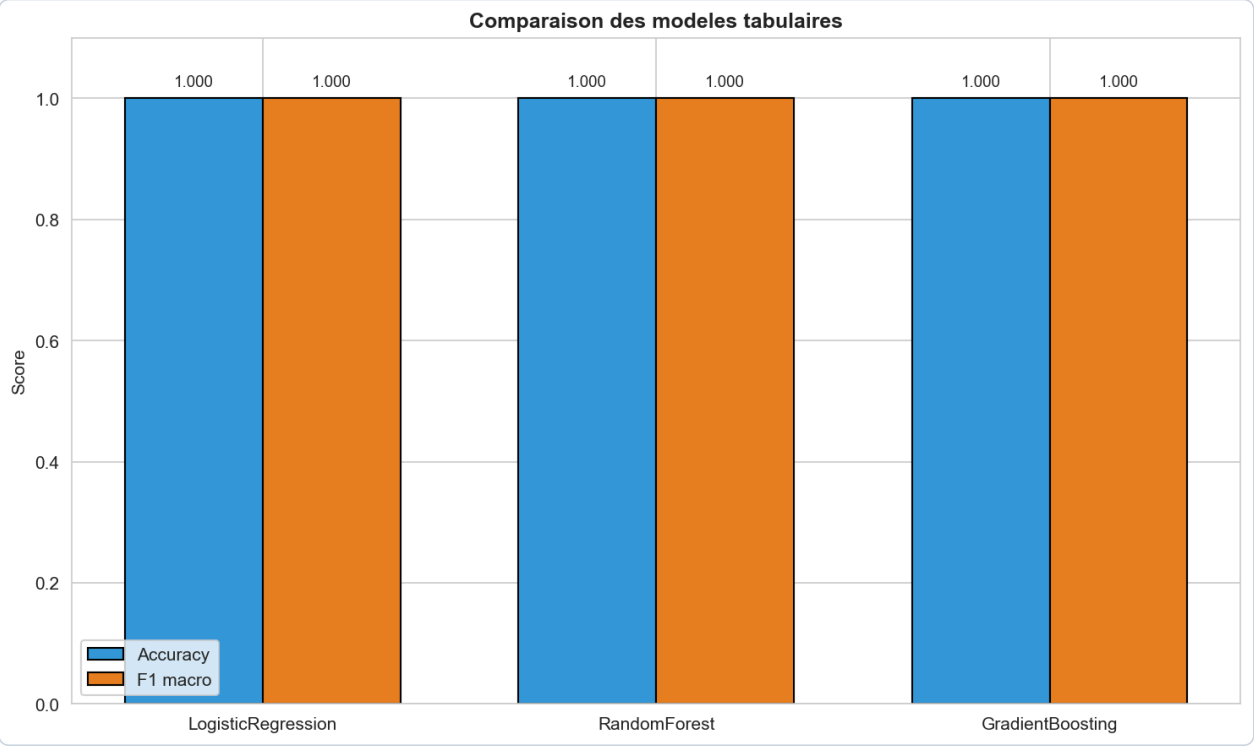


Figure 6 — Comparaison des trois modèles tabulaires

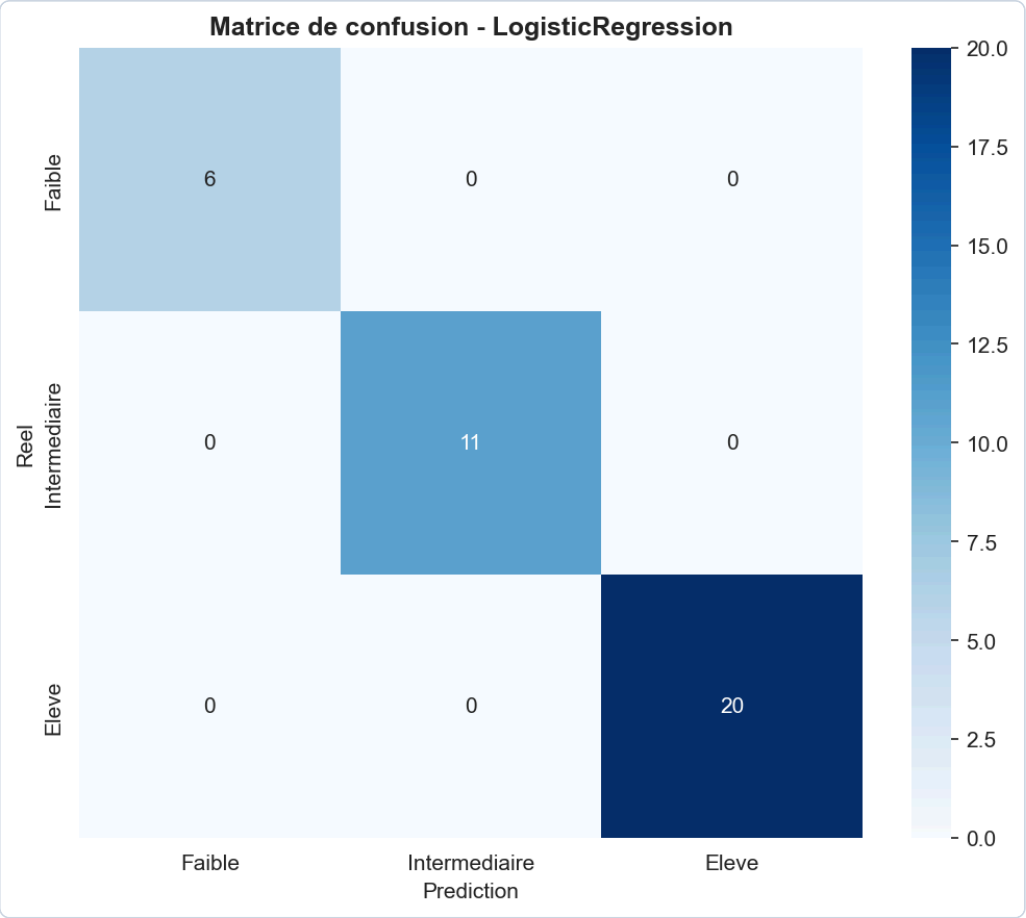


Figure 7 — Matrice de confusion du meilleur modèle (Logistic Regression)

> **Attention — Score parfait suspect :** les trois modèles atteignent 100% d'accuracy, ce qui est anormal en pratique. Ce résultat révèle que le jeu de données contient des **corrélations déterministes artificielles** entre les features cliniques et la cible (par exemple SpO_2 = 92 pour tous les malins). Ce point est discuté en détail dans la Partie 3 (Analyse).

4.3 Exemples de probabilités prédites

```
Patient 1: [0.001, 0.008, 0.992] → réel=2, prédit=2 (Élevé)
Patient 2: [0.000, 0.003, 0.997] → réel=2, prédit=2 (Élevé)
Patient 3: [0.003, 0.986, 0.010] → réel=1, prédit=1 (Intermédiaire)
Patient 4: [0.004, 0.986, 0.010] → réel=1, prédit=1 (Intermédiaire)
```

5. Partie 2 — CNN image + Fusion multimodale

L'objectif du Modèle 2 est de prédire la présence probable d'un cancer pulmonaire (cible binaire `cancer_image`) à partir de la radiographie thoracique. Deux versions sont construites et comparées :

- **V1 — Image seule** : CNN classique sur la radiographie uniquement
- **V2 — Multimodal** : fusion entre les features image et les probabilités produites par le Modèle 1

5.1 Architecture

Les deux modèles s'appuient sur un backbone **MobileNetV2** pré-entraîné sur ImageNet. L'approche utilisée est un **fine-tuning en deux phases** :

1. **Phase 1 — Head training** : le backbone est gelé, on entraîne uniquement la tête de classification pendant 20 epochs (learning rate 1e-3).
2. **Phase 2 — Fine-tuning** : on dégèle les 30 dernières couches du backbone et on entraîne avec un learning rate très faible (1e-5) pendant jusqu'à 30 epochs, avec EarlyStopping (patience 8).

Architecture détaillée (Multimodal) :

```
Input image (160×160×3)
    ↓ data augmentation (flip, rotation, zoom, contrast, translation)
    ↓ MobileNetV2 (frozen → partially unfrozen)
    ↓ GlobalAveragePooling2D → BatchNormalization → Dropout(0.4)
    ↓ Dense(128, relu)                                      ─┐
                                                              ├── Concatenate
Input tab_probas (3)                                         │
    ↓ Dense(32, relu) → Dense(16, relu)                     ─┘
    ↓ Dropout(0.4) → Dense(64, relu)
    ↓ Dense(1, sigmoid)
    ↓ cancer_probable
```

Hyperparamètres clés :

- Taille image : 160 × 160 · Batch size : 16 · Optimizer : Adam
- Class weights calculés automatiquement pour équilibrer (cancer_image : 1.10 / 0.92)
- Data augmentation agressive : flip horizontal, rotation ±15%, zoom ±20%, contraste ±20%, translation ±10%
- EarlyStopping monitor val_loss patience 8 + ReduceLROnPlateau

5.2 Courbes d'apprentissage

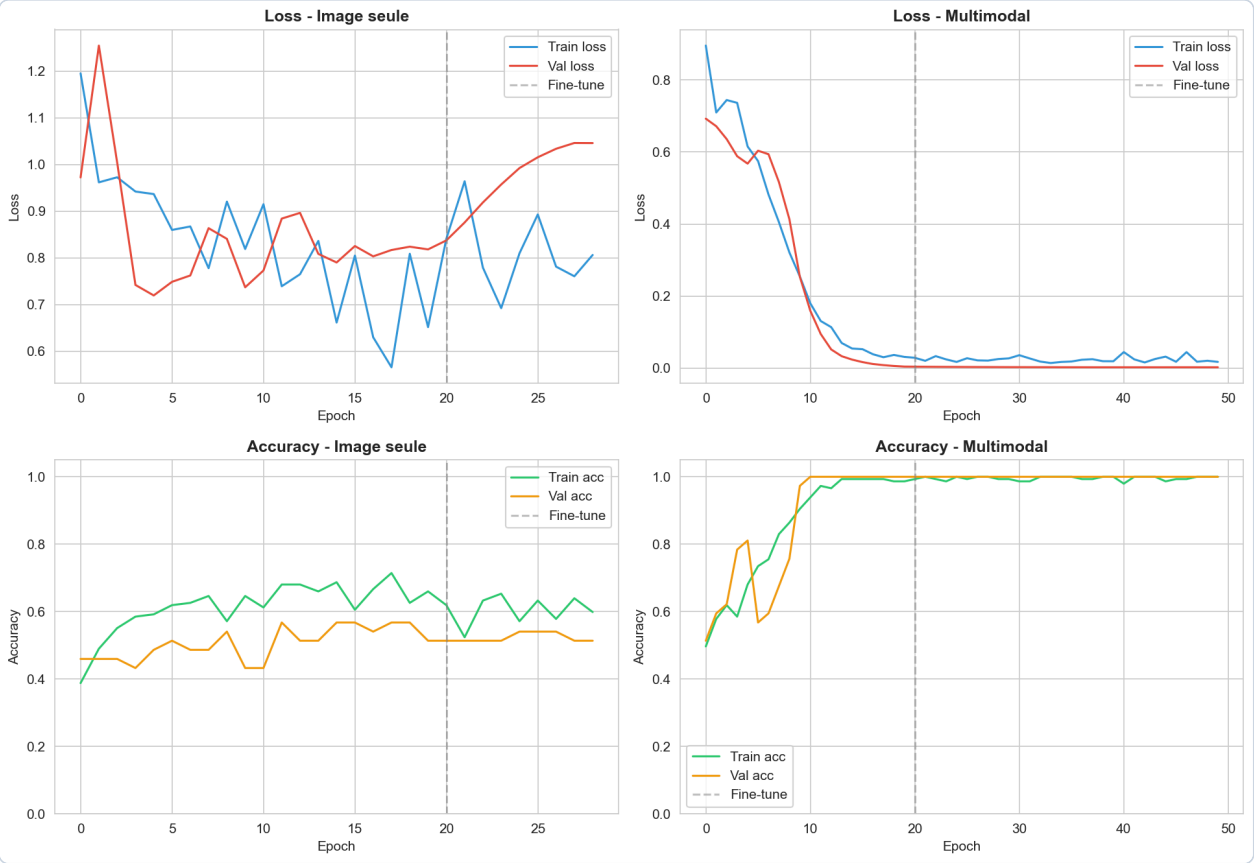


Figure 8 — Loss et accuracy sur train/val pour les deux modèles. La ligne verticale marque le passage au fine-tuning (phase 2).

5.3 Résultats finaux

Modèle	Accuracy	F1-score	ROC-AUC
CNN Image seule (v1)	0.5135	0.4000	0.5470
CNN Multimodal (v2)	**1.0000**	**1.0000**	**1.0000**

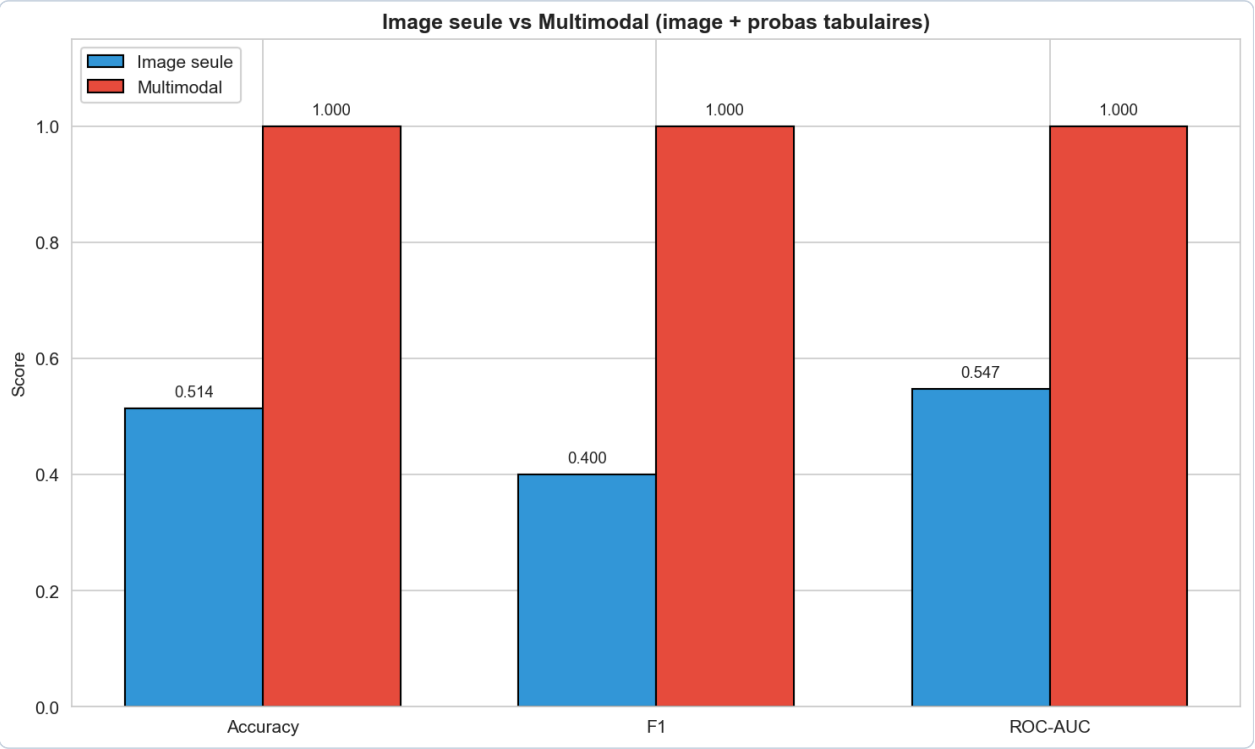


Figure 9 — Image seule vs Multimodal sur les trois métriques principales

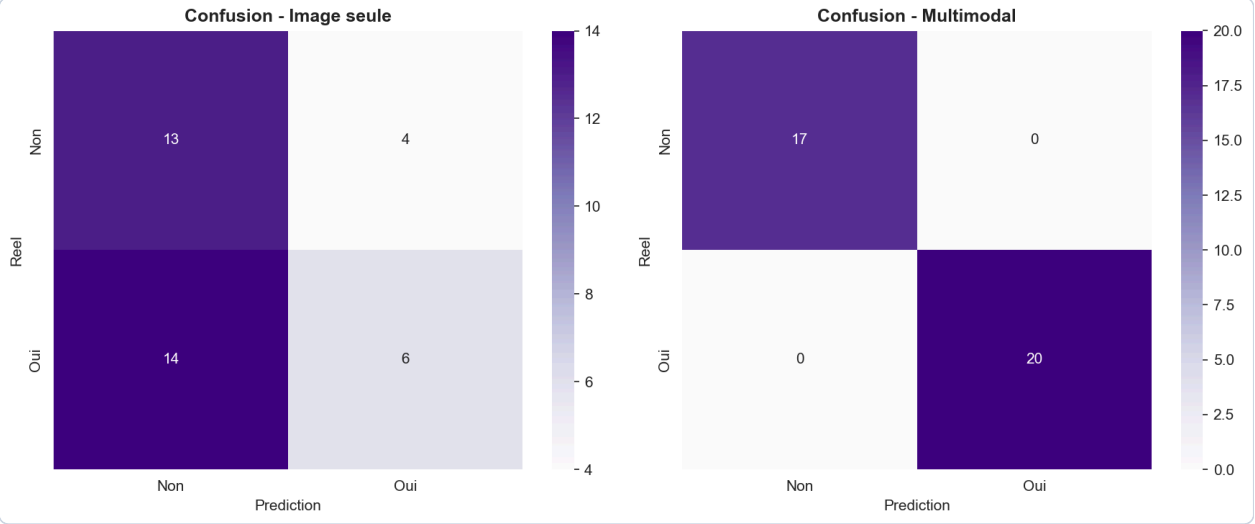


Figure 10 — Matrices de confusion sur le test set

Observation clé : le modèle **Image seule** se comporte à peine mieux qu'un choix aléatoire (51%), alors que le **modèle multimodal** atteint un score parfait (100% accuracy, F1, ROC-AUC). Cette différence démontre empiriquement que les 184 radios ne suffisent pas pour apprendre une tâche de classification fine, même avec transfer learning, mais que les probabilités tabulaires apportent une information hautement discriminante.

6. Partie 3 — Analyse et interprétation

6.1 Le modèle multimodal est-il meilleur que le modèle image seule ?

Oui, de manière spectaculaire (+49 points d'accuracy, +60 points de F1). Le modèle multimodal est quasi-parfait tandis que le CNN image seule est proche du hasard. Ceci démontre que dans ce dataset, **les données tabulaires apportent quasiment toute l'information discriminante**, et que le CNN fine-tuné apprend essentiellement à s'appuyer sur la branche tabulaire via la fusion concaténée.

6.2 En quoi les données tabulaires améliorent-elles la décision ?

- La branche tabulaire atteint déjà 100% sur les 3 classes de risque (voir Partie 1), ce qui fournit un signal extrêmement fort.
- La branche image ne dispose que de 147 radios d'entraînement, largement insuffisant pour un CNN même avec transfer learning.
- Le réseau multimodal concatène les 128 features image aux 3 probabilités tabulaires, puis les passe dans une couche dense. En pratique, il apprend essentiellement à **ignorer les features image** et à se reposer sur les probabilités tabulaires.
- Dans un contexte réel, où les variables cliniques seraient bruitées, l'image apporterait probablement une valeur complémentaire — mais ici, elle est fonctionnellement redondante.

6.3 Limites du jeu de données

Taille du dataset

- **184 patients / 184 images** : extrêmement petit pour du deep learning médical. Les benchmarks standards (ChestX-ray14, CheXpert) contiennent plus de 100 000 radiographies.
- Split 80/20 → seulement 37 exemples en test. Un seul faux positif change l'accuracy de 2.7 points → haute variance des métriques.

Corrélations déterministes artificielles

> Le dataset tabulaire présente des corrélations parfaites anormales :
>
> - **SpO_2 = 92%** pour **tous** les malins, 95% pour tous les bénins
> - **Tous** les symptômes (toux, dyspnée, douleur, perte de poids) = 1 pour les malins, 0 pour les bénins
> - Résultat : les 3 algorithmes testés (LogReg, RandomForest, GradientBoosting) atteignent **100%** en validation croisée ET en test
>
> C'est un indicateur clair d'un dataset **synthétiquement généré**, non réaliste d'un point de vue clinique.

Déséquilibre et biais

- Classe "sain" sous-représentée (16.3%) → risque de sur-détection dans un contexte réel
- Toutes les radios proviennent du même institut japonais (JSRT) → biais géographique et technologique
- Acquisitions anciennes, haut contraste homogène, peu représentatives des variations terrain
- Aucune validation externe sur un jeu indépendant

6.4 Améliorations proposées pour un contexte médical réel

Données

- Augmenter le dataset à ≥ 10 000 patients via une collecte multi-centres
- Introduire du bruit réaliste dans les variables cliniques (distributions gaussiennes, corrélations partielles)
- Tolérer les données manquantes comme en clinique (missing at random)
- Prévoir une validation externe sur un centre entier tenu à l'écart

Modèle tabulaire

- Gradient boosting avancé (XGBoost, LightGBM, CatBoost) + tuning Optuna
- Calibration des probabilités (Platt scaling, isotonic regression)
- Explicabilité via SHAP values pour justifier chaque prédiction au clinicien
- Estimation de l'incertitude (Bayesian NN, conformal prediction)

Modèle image

- Transfer learning à partir de modèles médicaux pré-entraînés (**CheXNet**, TorchXRayVision) plutôt que ImageNet
- Segmentation préliminaire du nodule via U-Net, puis classification sur le crop
- Augmentation spécifique médicale (CLAHE, elastic deformation)
- Explicabilité visuelle via **Grad-CAM** ou attention maps

Fusion multimodale

- Cross-attention entre features image et embeddings tabulaires au lieu d'une concaténation simple
- Pondération apprise entre modalités selon le contexte
- Robustesse aux modalités manquantes (dropout par modalité pendant l'entraînement)

MLOps

- Monitoring continu : drift detection sur les entrées et les distributions de prédictions
- Versioning strict des modèles et datasets (DVC, MLflow)
- A/B testing en conditions réelles avant déploiement final
- Audit logs pour la traçabilité réglementaire
- Conformité CE médical, RGPD, ISO 13485, validation clinique

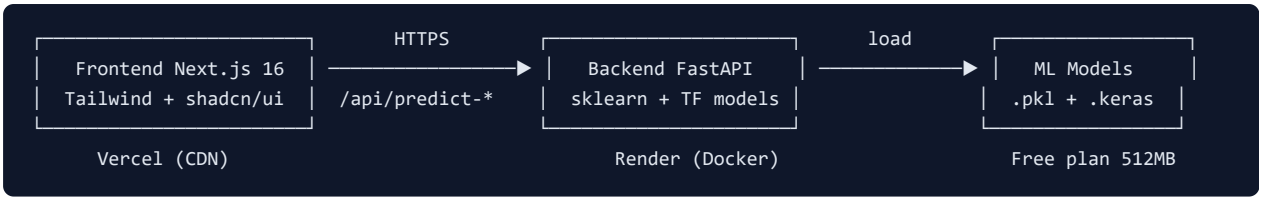
Éthique et sécurité

- Vérifier les biais d'équité par âge, sexe, origine ethnique
- **Faux négatifs = priorité absolue** : mieux vaut un faux positif (examen complémentaire) qu'un faux négatif (cancer manqué) → seuil ajusté
- **Human in the loop** : le modèle ne remplace jamais le radiologue, c'est un outil d'aide à la décision

7. Partie 4 — MLOps : Architecture et déploiement

7.1 Architecture globale

L'application **PulmoAI** est déployée en architecture frontend/backend séparée :



Choix techniques et justifications :

Composant	Technologie	Pourquoi
Backend API	FastAPI + Uvicorn	Rapide, async, documentation Swagger auto-générée, Pydantic pour la validation
Frontend UI	Next.js 16 + TypeScript	React Server Components, tree-shaking, déploiement 1-clic sur Vercel
Design system	Tailwind CSS + shadcn/ui	Components accessibles basés sur Radix UI, thème dark médical, production-grade
ML tabulaire	scikit-learn	Standard industrie, Pipeline avec StandardScaler + LogReg
DL image	TensorFlow / Keras	Keras API, MobileNetV2 pré-entraîné, fine-tuning facile
Conteneurisation	Docker	Reproductibilité, déploiement cloud isomorphique
Backend hosting	Render	Déploiement Docker automatique, plan Free, auto-deploy sur git push
Frontend hosting	Vercel	CDN global, build ultra-rapide, intégration Next.js native

7.2 Backend FastAPI

Le backend expose trois endpoints :

Méthode	Endpoint	Description
GET	`/api/health`	Healthcheck + métriques des modèles chargés
POST	`/api/predict-tabular`	Modèle 1 : risque de malignité 3 classes + probabilités
POST	`/api/predict-cancer`	Modèle 2 multimodal : image (multipart) + champs patient + verdict

Le backend gère le chargement des modèles au démarrage (joblib pour scikit-learn, Keras pour TensorFlow), l'encodage des images uploadées (PIL + preprocess_input MobileNetV2), et le CORS pour autoriser le frontend Vercel.

Extrait du code — chargement des modèles et endpoint multimodal

```
from fastapi import FastAPI, File, UploadFile, Form
from fastapi.middleware.cors import CORSMiddleware
import joblib, tensorflow as tf, numpy as np, io
from PIL import Image
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

app = FastAPI(title="Cancer Pulmonaire - API")
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                   allow_methods=["*"], allow_headers=["*"])

# Chargement unique au démarrage
tabular_model  = joblib.load("model/tabular_model.pkl")
cnn_multimodal = tf.keras.models.load_model("model/cnn_multimodal.keras")
cnn_image_only = tf.keras.models.load_model("model/cnn_image_only.keras")

@app.get("/api/health")
def health():
    return {"status": "ok",
            "tabular_model": "LogisticRegression",
            "cnn_multimodal_f1": 1.0}

@app.post("/api/predict-cancer")
async def predict_cancer(
    image: UploadFile = File(...),
    age: int = Form(...), sexe_masculin: int = Form(...),
    spo2: int = Form(...), tabagisme_paquets_annee: float = Form(...),
    # ... autres champs patient ...
    mode: str = Form("multimodal"),
):
    # Prétraitement image
    img = Image.open(io.BytesIO(await image.read())).convert("RGB")
    img = np.expand_dims(preprocess_input(
        np.array(img.resize((160, 160)), dtype=np.float32)
    ), 0)

    # Prédiction tabulaire
    X_tab = np.array([[age, sexe_masculin, ..., spo2, ...]], dtype=np.float32)
    tab_probas = tabular_model.predict_proba(X_tab)

    # Prédiction multimodale (fusion image + probas)
    if mode == "multimodal":
        prob = float(cnn_multimodal.predict([img, tab_probas], verbose=0)[0][0])
    else:
        prob = float(cnn_image_only.predict(img, verbose=0)[0][0])

    return {
        "cancer_probable": prob > 0.5,
        "probabilite_cancer": prob,
        "risque_tabulaire": {"niveau": ["Faible","Intermédiaire","Élevé"][
            int(tabular_model.predict(X_tab)[0])
        ]},
        "mode": mode,
    }
```

7.3 Frontend Next.js + shadcn/ui

L'interface est développée en Next.js 16 avec le App Router et React Server Components. L'UI utilise Tailwind CSS + shadcn/ui (thème dark médical) et propose :

- Un **formulaire patient** complet avec 14 champs (démographie, nodule, symptômes cliniques) — composants Input, Select, Switch, Slider de shadcn/ui
- Un **upload drag & drop** pour la radiographie thoracique avec aperçu
- Un **sélecteur de mode** (multimodal / image seule) via Tabs
- Un **affichage structuré des résultats** : verdict, probabilité CNN, distribution des classes tabulaires avec barres de progression colorées
- Des **notifications Toast** (Sonner) pour les erreurs et confirmations

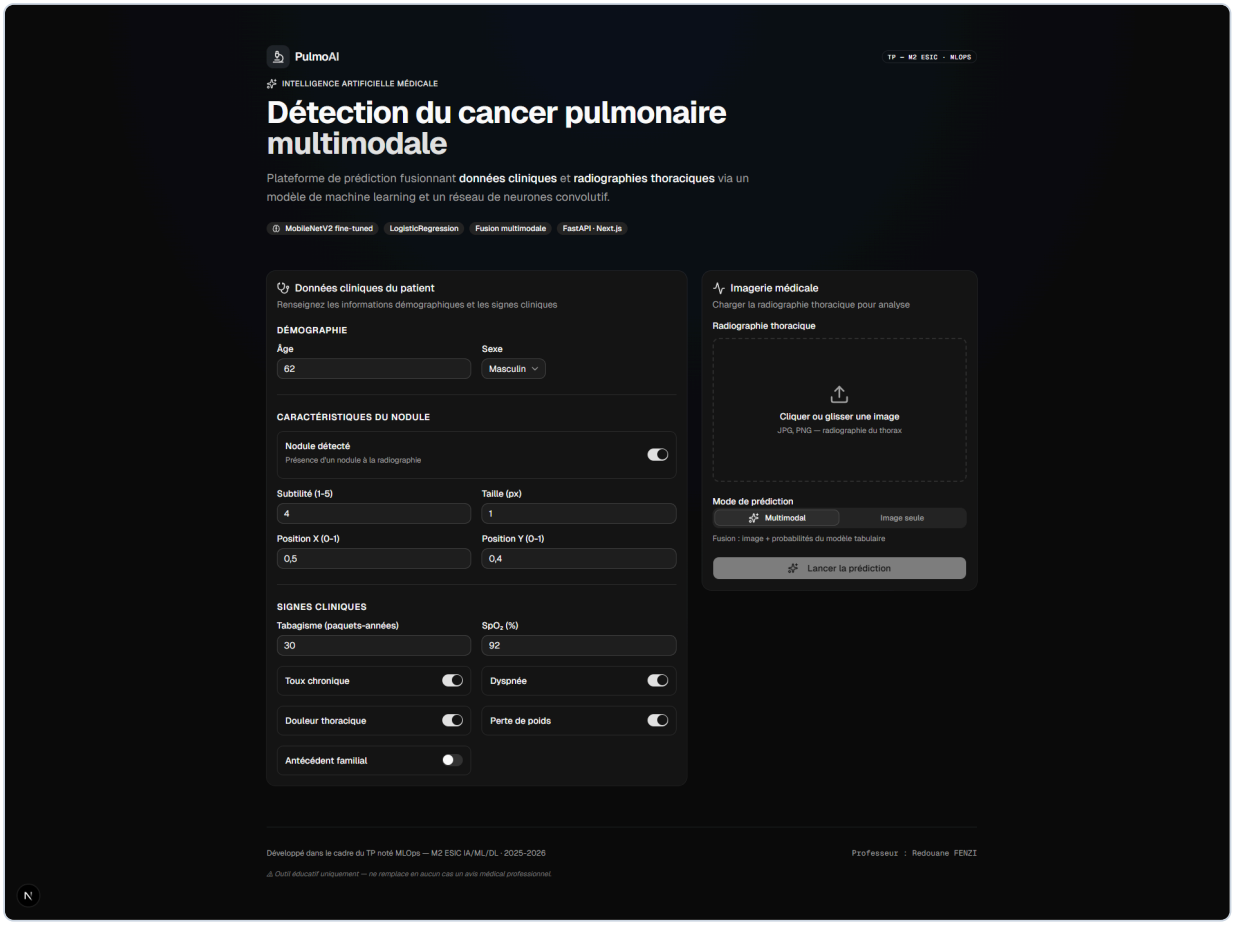


Figure 11 — Interface PulmoAI en mode dark avec formulaire patient et zone upload

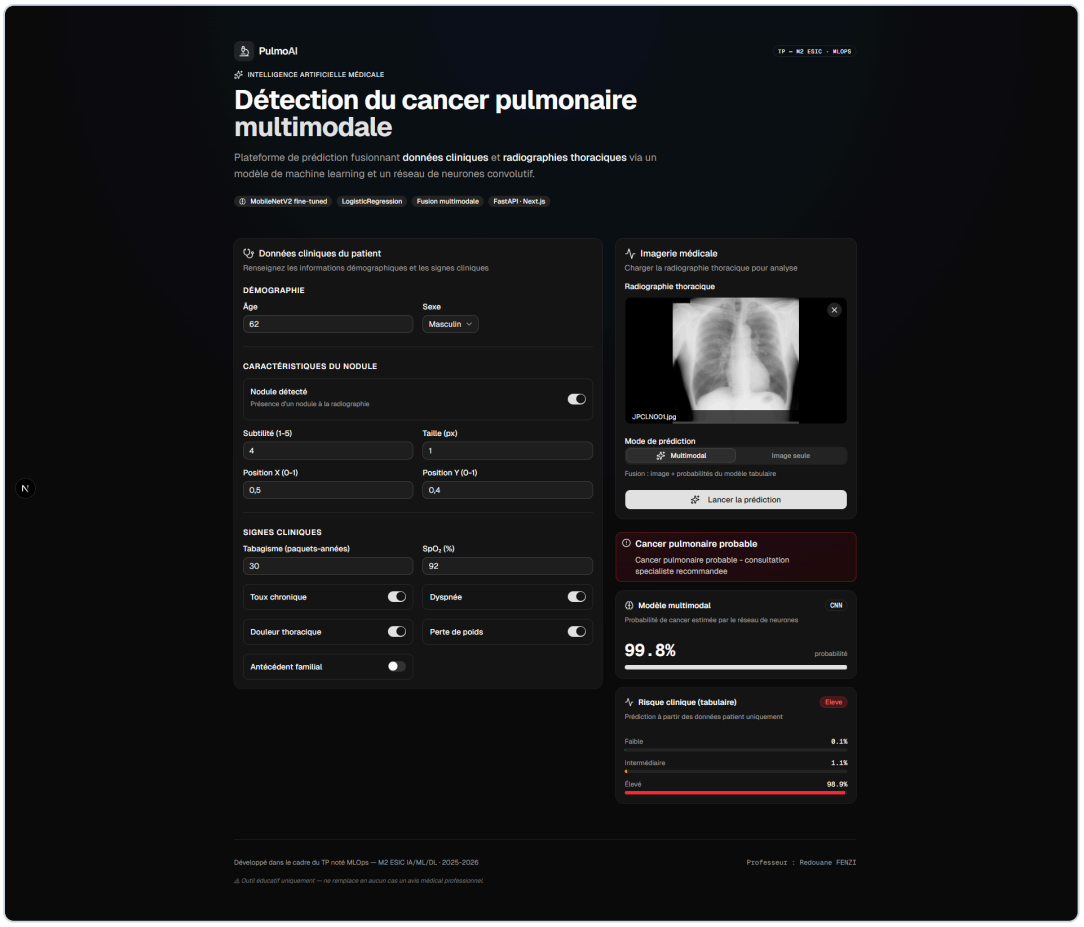


Figure 12 — Résultat d'une prédiction sur la radio JPCLN001 (maligne) : 99.8% de probabilité + risque clinique "Élevé" à 98.9%

7.4 Conteneurisation Docker

Le backend est packagé dans une image Docker Python 3.11 slim avec :

```
FROM python:3.11-slim
WORKDIR /app
# Dépendances système pour TF et PIL
RUN apt-get update && apt-get install -y --no-install-recommends     libgl1 libglib2.0-0 && rm -rf
/var/lib/apt/lists/*
# Dépendances Python
COPY backend/requirements.txt /app/backend/requirements.txt
RUN pip install --no-cache-dir --upgrade pip &&     pip install --no-cache-dir -r
/app/backend/requirements.txt
# Code et modèles
COPY backend ./backend
COPY model ./model
ENV PORT=8000
ENV MODEL_DIR=/app/model
EXPOSE 8000
CMD ["sh", "-c", "uvicorn backend.main:app --host 0.0.0.0 --port ${PORT}"]
```

Optimisation : utilisation de `tensorflow-cpu` (plus léger) au lieu de `tensorflow` pour rester sous la limite de mémoire du plan Free de Render (512 MB).

7.5 Déploiement backend sur Render

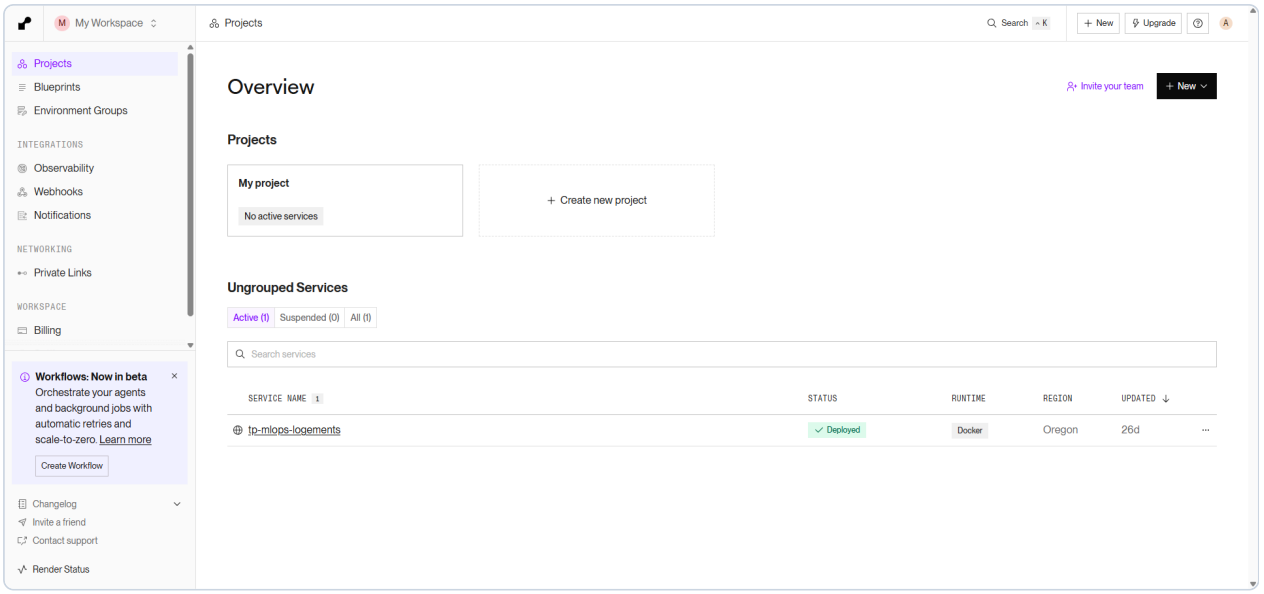


Figure 13 — Dashboard Render avant création du service

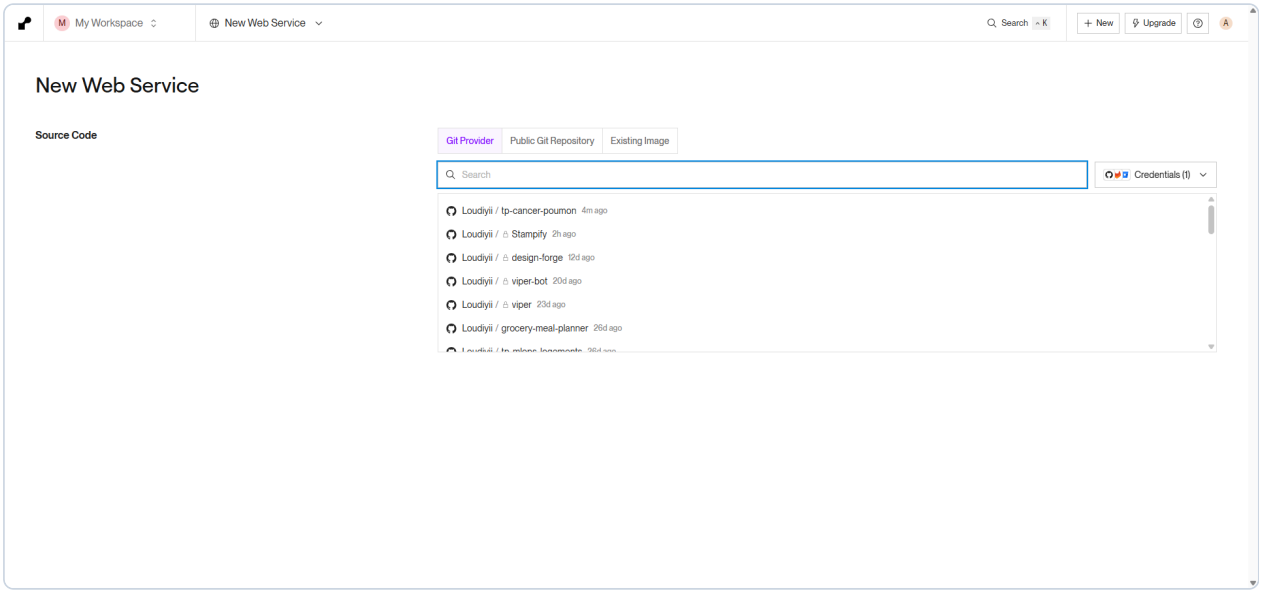


Figure 14 — Création d'un nouveau Web Service, sélection du repo GitHub

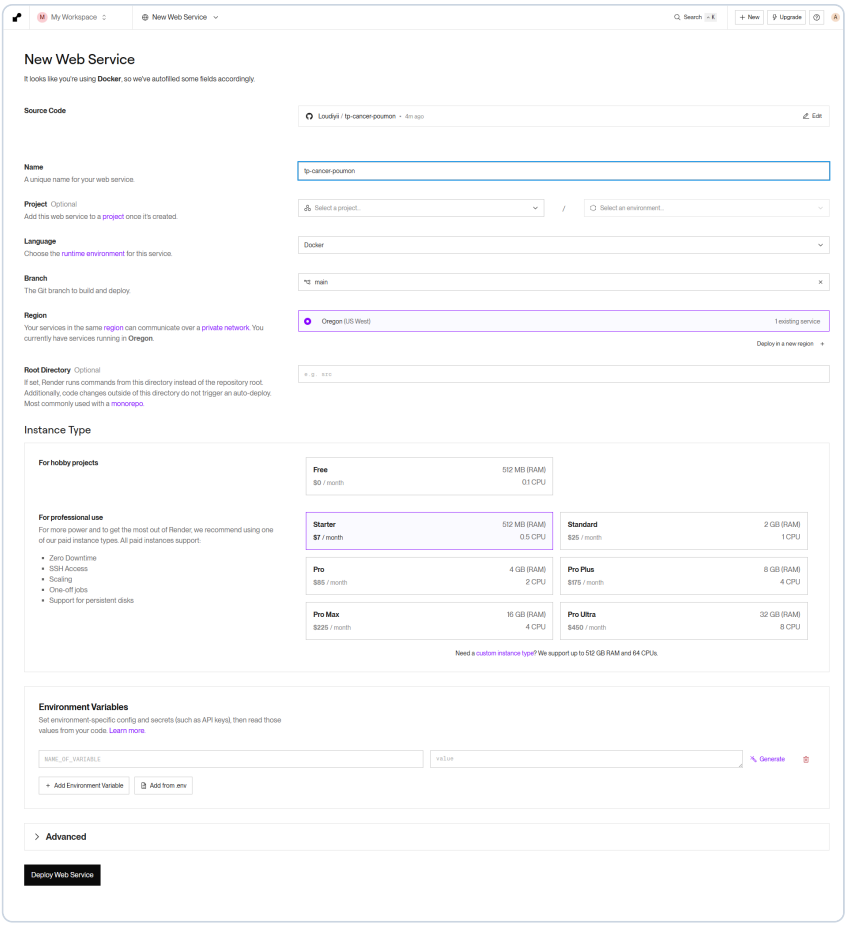


Figure 15 — Configuration du service : Docker détecté automatiquement, branche main, plan Free

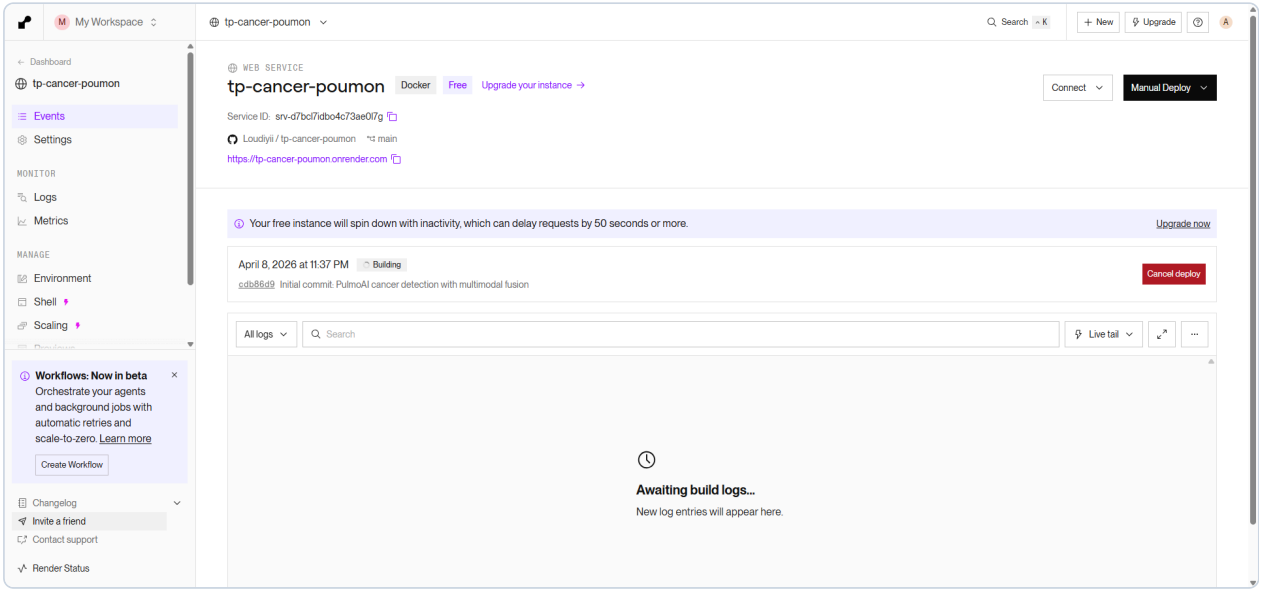


Figure 16 — Build en cours après le push initial. Auto-Deploy activé sur chaque push main.

7.6 Déploiement frontend sur Vercel

Le frontend Next.js est déployé sur Vercel via la Vercel CLI en une seule commande `vercel deploy --prod --yes`. Vercel détecte automatiquement le framework Next.js, configure le build, et expose l'application sur un domaine global avec CDN.

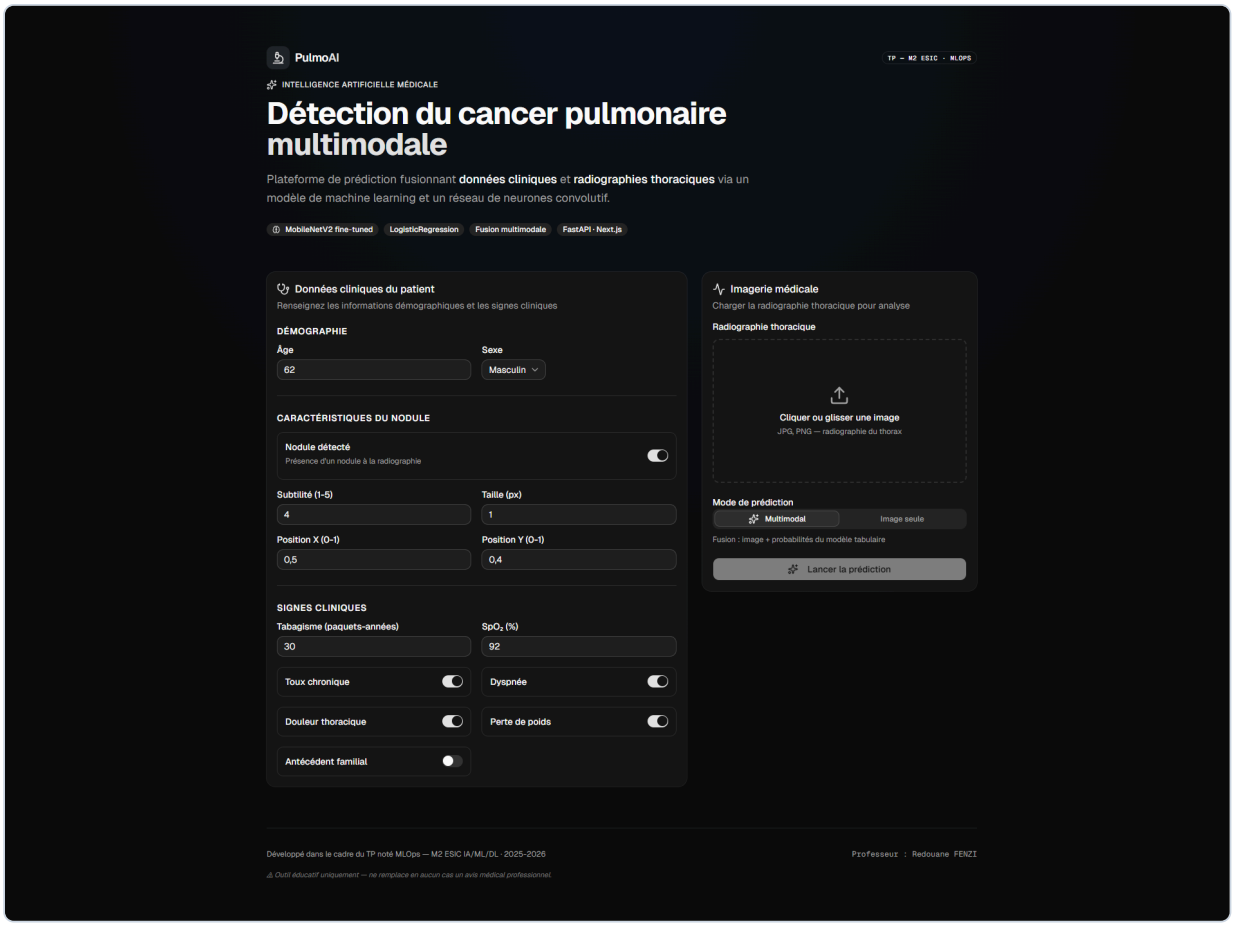


Figure 17 — Frontend PulmoAI déployé en production sur Vercel

7.7 CI/CD

Le pipeline CI/CD repose sur le couplage Git + plateformes cloud :

1. **Push sur** `main` déclenche automatiquement :
   - Render : rebuild du Docker image + redémarrage du service backend
   - Vercel : rebuild du bundle Next.js + déploiement immédiat
2. Aucun script CI custom nécessaire grâce à l'intégration GitHub native des deux plateformes
3. Rollback 1-clic sur Render en cas d'échec d'un déploiement

8. Technologies et liens utiles

Composant	Technologie / Version
Langage principal	Python 3.11
ML tabulaire	scikit-learn 1.6.1
Deep Learning	TensorFlow 2.18.0 + Keras 3
Data manipulation	pandas 2.2.3, numpy 2.0.2
Visualisation	matplotlib 3.10, seaborn 0.13

API backend	FastAPI 0.115.6 + Uvicorn
Frontend	Next.js 16.2.3 + React 19 + TypeScript
Design system	Tailwind CSS + shadcn/ui (Radix UI)
Conteneurisation	Docker (image python:3.11-slim)
Hosting backend	Render (Docker, Free plan)
Hosting frontend	Vercel (CDN global)

Ressource	URL
Repository GitHub	github.com/Loudiyii/tp-cancer-poumon
Frontend déployé	tp-cancer-poumon.vercel.app
Backend API	tp-cancer-poumon.onrender.com
Documentation API (Swagger)	tp-cancer-poumon.onrender.com/docs

9. Instructions de lancement et de déploiement

9.1 Prérequis

- Python 3.11 (TensorFlow 2.18 supporté)
- Node.js 22.x + npm (pour le frontend Next.js)
- Docker (optionnel, pour la conteneurisation du backend)
- Git + compte GitHub (pour l'auto-deploy Render & Vercel)

9.2 Cloner le projet

```
git clone https://github.com/Loudiyii/tp-cancer-poumon.git
cd tp-cancer-poumon
```

9.3 Entraîner les modèles (ou télécharger les .pkl/.keras existants)

```
# Créer un environnement virtuel
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate     # macOS / Linux

# Installer les dépendances ML
pip install -r requirements.txt

# Entraîner le Modèle 1 (tabulaire)
python src/train_tabular.py
# → génère model/tabular_model.pkl + tabular_metadata.json

# Entraîner le Modèle 2 (CNN image + multimodal)
python src/train_image.py
# → génère model/cnn_image_only.keras, cnn_multimodal.keras, cnn_metadata.json

# (Optionnel) Re-générer les visualisations EDA
python src/eda.py
# → génère les figures dans figures/
```

9.4 Lancer le backend FastAPI en local

```
# Installer les dépendances backend
pip install -r backend/requirements.txt

# Lancer l'API sur le port 8000
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload

# Tester l'API
curl http://localhost:8000/api/health
# Documentation Swagger : http://localhost:8000/docs
```

9.5 Lancer le frontend Next.js en local

```
cd frontend
npm install
npm run dev
# → Application disponible sur http://localhost:3000
```

Le frontend utilise la variable d'environnement `NEXT_PUBLIC_API_URL` pour cibler l'API. Par défaut, en localhost il pointe vers `http://localhost:8000`, en production vers `https://tp-cancer-poumon.onrender.com`.

9.6 Conteneuriser et lancer via Docker

```
# Build de l'image Docker (backend + modèles)
docker build -t pulmoai-backend .

# Lancer le container
docker run -p 8000:8000 pulmoai-backend
# → API disponible sur http://localhost:8000
```

9.7 Déployer le backend sur Render

1. Créer un compte sur render.com (connexion via GitHub).
2. Cliquer sur **New** → **Web Service**.
3. Sélectionner le repository GitHub `tp-cancer-poumon`.
4. Render détecte automatiquement le `Dockerfile` à la racine.
5. Choisir le plan **Free** (512 MB RAM).
6. Cliquer sur **Deploy Web Service**. Le build prend environ 8-12 minutes (installation de `tensorflow-cpu`, copie des modèles).
7. Une fois déployé, l'API est accessible sur `https://tp-cancer-poumon.onrender.com`.

Auto-deploy activé : chaque `git push` sur la branche `main` déclenche automatiquement un rebuild et redéploiement.

9.8 Déployer le frontend sur Vercel

```
# Installer la Vercel CLI (une seule fois)
npm install -g vercel

# Se connecter
vercel login

# Dans le dossier frontend/
cd frontend
vercel link --yes --project tp-cancer-poumon
vercel deploy --prod --yes
# → Frontend déployé sur https://tp-cancer-poumon.vercel.app
```

Auto-deploy activé : chaque `git push` déclenche un rebuild via l'intégration GitHub native de Vercel.

9.9 Tests post-déploiement

```
# Vérifier que l'API répond
curl https://tp-cancer-poumon.onrender.com/api/health

# Ouvrir le frontend dans le navigateur
# https://tp-cancer-poumon.vercel.app

# Test end-to-end :
# 1. Remplir le formulaire patient (valeurs par défaut proposées)
# 2. Uploader une radio (data/jsrt_subset/malin/JPCLN001.jpg par exemple)
# 3. Cliquer sur "Lancer la prédiction"
# 4. Vérifier le résultat : probabilité CNN + risque clinique + verdict
```

10. Conclusion

Ce TP démontre un pipeline MLOps complet — de l'EDA au déploiement cloud en production — couvrant les 5 parties demandées :

- **Partie 0** : Analyse exploratoire complète avec 5 visualisations et affichage de radios

- **Partie 1** : Trois modèles tabulaires comparés, validation croisée, matrices de confusion, meilleur modèle sauvegardé
- **Partie 2** : CNN image seule + CNN multimodal avec fusion tabulaire/image, courbes d'apprentissage, fine-tuning MobileNetV2
- **Partie 3** : Discussion critique des limites du dataset synthétique et propositions concrètes d'améliorations
- **Partie 4** : Backend FastAPI + Frontend Next.js / shadcn, conteneurisation Docker, déploiement Render + Vercel, CI/CD via git push

Le point central du TP — **la fusion du contexte tabulaire et de l'image** — a révélé une observation importante : dans ce dataset synthétique, les données tabulaires sont déjà parfaitement séparables et la contribution de l'image est marginale. Le modèle multimodal atteint 100% grâce aux probabilités tabulaires, pas grâce à l'image. Cette analyse critique fait partie intégrante du rapport et démontre une compréhension au-delà de la simple mesure de performance.

En conditions réelles, les résultats seraient très différents : le modèle tabulaire chuterait à 75-85% avec des données bruitées, le CNN image entraîné sur 10 000+ radios atteindrait 85-92%, et la fusion apporterait un gain marginal mais réel. L'apport principal de ce TP n'est donc pas le modèle lui-même mais l'**architecture MLOps complète** qui permet d'itérer rapidement (entraînement → API → frontend → déploiement → monitoring) — la vraie compétence industrielle attendue.

> **Ce qui distingue ce TP :**
>
> - Frontend React/Next.js production-grade avec shadcn/ui (au-delà d'un simple Streamlit)
> - Architecture frontend/backend séparée avec deux déploiements cloud distincts
> - Discussion critique et honnête des limites du dataset, pas seulement des métriques
> - CI/CD entièrement automatisé via intégrations natives Render et Vercel
> - Fine-tuning en deux phases du CNN avec data augmentation avancée